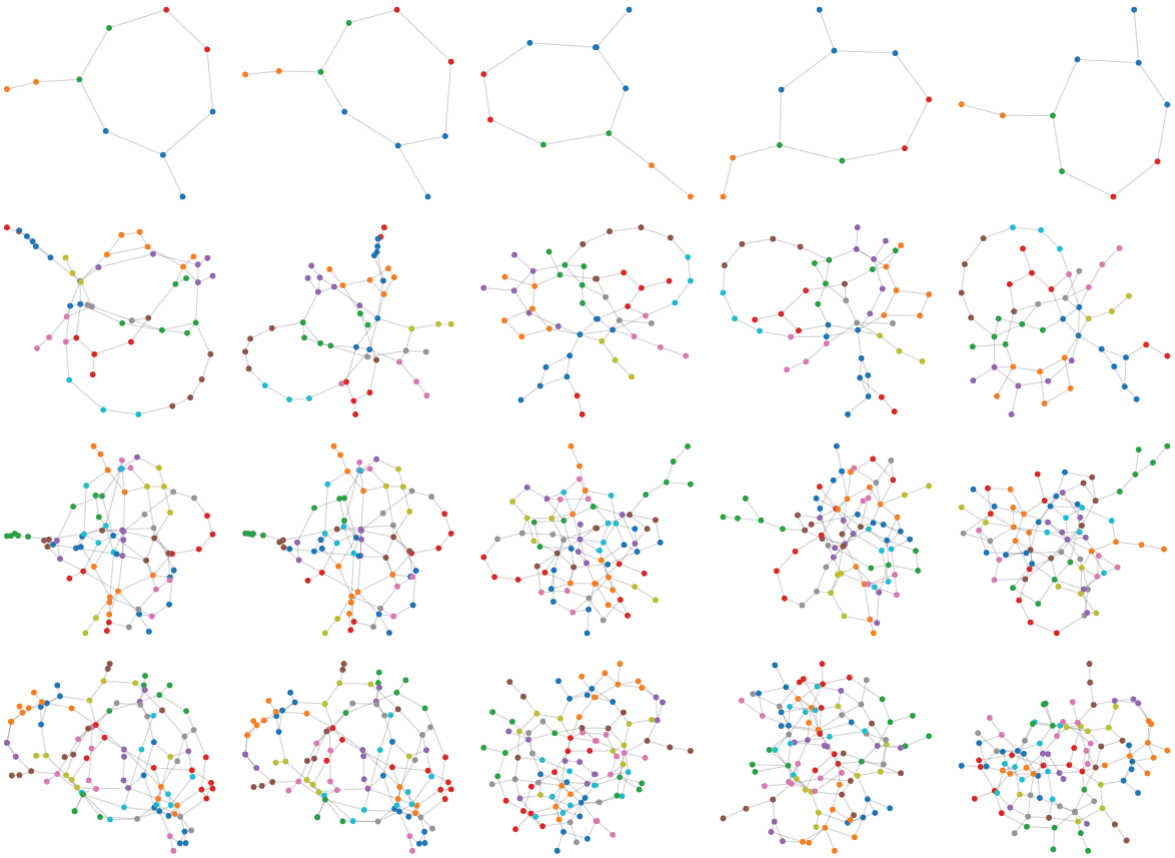
Plongement de graphes par modèles d'apprentissage profond et descente de gradient directement en espace écran avec des approches de dessin différentiable

Romain Giot <Romain.Giot@u-bordeaux.fr>



Contexte

Une partie des travaux de recherche menés par l'équipe BKB du LaBRI porte sur la projection de données en haute dimension dans un espace plus restreint, souvent continu en deux dimensions. Ces données peuvent être de nature tabulaire ou représenter des graphes. Les méthodes classiques de plongement et de dessin de graphes optimisent généralement des critères de qualité dans un espace géométrique, alors que le rendu final s'effectue dans un espace discret constitué des pixels de l'écran. Cette dissociation entre l'espace de modélisation et l'espace d'affichage pose un problème fondamental : **les contraintes de visibilité, de superposition et d'occupation du plan définies dans le domaine continu ne sont pas nécessairement conservées après rasterisation**.

L'objectif de ce projet est de proposer une alternative à cette approche traditionnelle en construisant un cadre d'**optimisation directement compatible avec l'espace écran**. Le but n'est pas seulement de modifier la forme de rendu, mais d'intégrer les contraintes visuelles de la sortie affichée dans le cœur même du processus d'optimisation. Cette perspective ouvre des questions théoriques et méthodologiques au carrefour du **dessin de graphes**, du **rendu différentiable** et de l'**apprentissage profond**.

Projet

Le projet consiste à concevoir une bibliothèque de rendu et d'optimisation capable d'ajuster la disposition des nœuds d'un graphe directement dans l'espace écran, au lieu de se limiter à un espace géométrique intermédiaire. L'étude s'inspire des méthodes classiques de dessin, tout en remplaçant progressivement les fonctions de coût traditionnelles (en espace géométrique) par des formulations calculées dans l'espace écran. Le dessin des nœuds et des arêtes est représenté à l'aide de fonctions de distance signées (2d Signed Distance Function), ce qui permet d'obtenir une description différentiable et exploitable par des algorithmes de descente de gradient.

Cette démarche est naturellement exploratoire. Elle laisse une large place à l'innovation méthodologique et encourage la proposition de nouveaux critères d'évaluation adaptés au rendu visuel. Les données d'entrée comprennent la liste des arêtes, la taille des nœuds, ainsi que des attributs optionnels tels que la largeur des arêtes ou les couleurs des nœuds et des arêtes. Les données de sortie consistent en un ensemble de coordonnées 2D pour chaque nœud, éventuellement complété par une représentation rasterisée du graphe. Le problème central est de savoir si des fonctions de coût définies dans l'espace écran permettent d'obtenir des mises en page plus conformes aux contraintes visuelles du rendu final que des approches optimisées uniquement dans un espace géométrique.

Deux axes de recherche doivent être menés en parallèle. Le premier concerne la définition de fonctions de coût spécifiques au rendu écran, conçues pour capturer des critères visuels tels que les recouvrements, les collisions, la lisibilité et la cohérence du dessin. Le second consiste à développer un cadre de transformation des données capable d'optimiser directement la position des nœuds en minimisant ces fonctions de coût pour une donnée donnée. Une troisième perspective, plus ambitieuse, consiste à apprendre un modèle profond sur une base de graphes afin de générer automatiquement un dessin à partir d'une structure de graphe donnée.

Les méthodes de référence qui inspirent ce travail sont à la fois issues des approches classiques de dessin de graphes, et des récents développements en apprentissage profond. Parmi les méthodes d'optimisation, on trouve $(GD)^2$ ainsi que les approches fondées sur la descente de gradient stochastique. Parmi les méthodes basées sur des modèles, on peut citer DL4GD, DeepGD et SmartGD. Les fonctions de distance signées, telles que celles décrites dans les travaux de Iñigo Quilez, permettent de dessiner les formes géométriques dans une ou plusieurs cartes d'activation. À partir de ces représentations, il devient possible de calculer des coûts directement en espace écran, de manière compatible avec l'optimisation par descente de gradient. L'objectif est alors de transposer, dans le domaine du pixel, les fonctions de coût qui sont traditionnellement définies dans le domaine géométrique. A noter que la quantité de calcul à réalisée (et de mémoire à utiliser) sera importante et de l'ordre de (nombre de noeuds + nombre d'arêtes)*(largeur* hauteur de l'image) ; on privilégiera donc des essais sur des petits graphes de la base Rome.

Travail à réaliser

Le projet repose sur le développement d'un pipeline de dessin différentiable pour les graphes, implémenté en Python avec PyTorch. La première étape consiste à mettre en place une procédure de rasterisation à partir d'un plongement et d'attributs fourni, en s'aidant de fonctions de distance signées (cad **fonction de rendu différentiable**). Cette représentation, ou les données intermédiaires pour la produire, est ensuite utilisée pour définir des fonctions de coût calculées directement dans l'espace écran, en tenant compte des

contraintes visuelles de la sortie finale (cad **fonction de coût perceptuelle différentiable**). La seconde étape consiste à construire un processus d'optimisation par descente de gradient permettant de modifier les positions des nœuds afin de minimiser ces coûts (cad **plongement par descente de gradient**). Une approche par modèle profond est également envisageable (cad **plongement par réseau de neurones profond**). Enfin, le système est évalué par comparaison avec des méthodes classiques de dessin de graphes afin de mesurer les gains ou pertes de qualité associés à l'optimisation en espace écran.

Le projet est conçu comme un travail de recherche appliquée et de preuve de concept . Il ne vise pas seulement à reproduire des méthodes existantes, mais à explorer la validité d'une reformulation du problème du dessin de graphes dans un espace plus proche du rendu final. En fonction des résultats obtenus, l'encadrement pourra orienter le projet vers une variante plus théorique, plus algorithmique ou plus centrée sur l'apprentissage profond. Nous disposons également d'exemples de code.

Pile technique

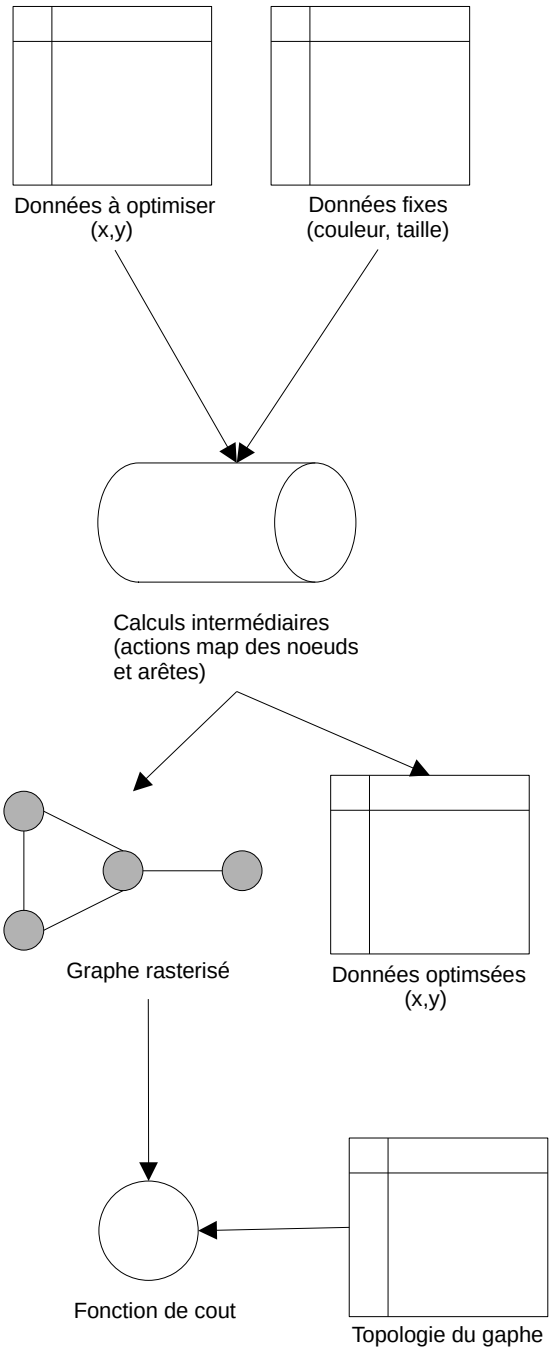
Le développement est mené en Python, avec PyTorch comme principal cadre d'implémentation. Le projet repose sur des outils de gestion de version standard, notamment git et GitLab, dans le cadre d'un travail reproductible et collaboratif.

Bibliographie

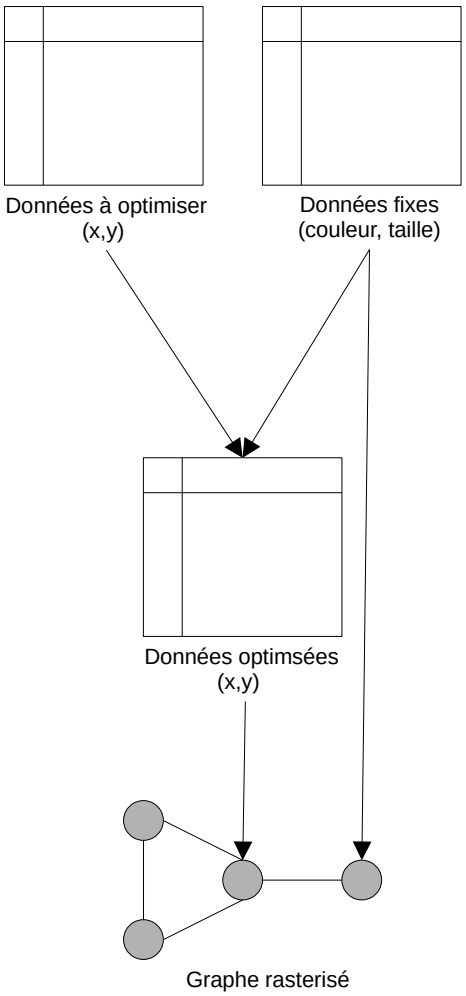
- Giovannangeli, L., Lalanne, F., Auber, D., Giot, R., & Bourqui, R. (2022). Toward efficient deep learning for graph drawing (DL4GD). IEEE Transactions on Visualization and Computer Graphics, 30(2), 1516-1532.
- Zhang, S., Xu, R., Zhang, Q., Quan, Y., & Liu, Q. (2024). DeepFD: a deep learning approach to fast generate force-directed layout for large graphs. Journal of Visualization, 1-16.
- Giovannangeli, L., Lalanne, F., Giot, R., & Bourqui, R. (2022, September). FORBID: Fast Overlap Removal By stochastic gradient descent for Graph Drawing. In International Symposium on Graph Drawing and Network Visualization (pp. 61-76). Cham: Springer International Publishing.
- Wang, X., Yen, K., Hu, Y., & Shen, H. W. (2023). SmartGD: A GAN-Based Graph Drawing Framework for Diverse Aesthetic Goals. IEEE Transactions on Visualization and Computer Graphics.
- Ahmed, Reyan, et al. "Graph drawing via gradient descent, (GD) 2." International Symposium on Graph Drawing and Network Visualization. Cham: Springer International Publishing, 2020.
- Zheng, Jonathan X., Samraat Pawar, and Dan FM Goodman. "Graph drawing by stochastic gradient descent." IEEE transactions on visualization and computer graphics 25.9 (2018): 2738-2748.
- Inigo Quilez, 2D distance and gradient functions - 2019,https://iquilezles.org/articles/distgradfunctions2d/
- https://pytorch.org/

Annexe

Illustration de l'inférence en utilisant l'optimistion directe pour un graphe ou le dessin par réseau de neurones profond.



Optimisation par descente de gradient pour un graphe



Inférence avec réseau de neurones profond (après apprentissage) pour un graphe